

Chapitre 15. Fiche-cours : algorithme de Hörner.

Données :

- $P = a_n X^n + a_{n-1} X^{n-1} + \dots + a_1 X + a_0$ un polynôme.
En programmation, P sera stocké sous la forme d'une liste `coeff=[a0,...,an]` (attention, liste de longueur $n+1$).
- x un réel ou un complexe.

But de la méthode : Calculer $P(x)$ plus efficacement que par la méthode "naturelle".

Question 1 : Combien ce calcul nécessite-t-il d'opération élémentaires (produit et sommes) par la méthode naïve, basée sur l'écriture $P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$?

Il y a la somme et le terme $a_k x^k$ demande k multiplication
d'où $\sum_{k=0}^n \frac{n(n+1)}{2}$ multiplication $n + \frac{n(n+1)}{2} = O(n^2)$.

Présentation de l'algorithme :

- On pose au départ $q_0 = a_n$
- Puis on calcule $q_1 = q_0 x + a_{n-1}$
- Puis on calcule $q_2 = q_1 x + a_{n-2}$
- Puis on calcule $q_3 = q_2 x + a_{n-3}$
- ... jusqu'à q_n .

Ainsi, on construit une suite finie :

$$\begin{cases} q_0 = a_n \\ \forall k \in \{1, \dots, n\}, q_k = q_{k-1} x + a_{n-k} \end{cases}$$

Question 2 :

Simplifier au fur et à mesure...

$$q_1 = a_n x + a_{n-1}$$

$$q_2 = a_n x^2 + a_{n-1} x + a_{n-2}$$

$$q_3 = a_n x^3 + a_{n-1} x^2 + a_{n-2} x + a_{n-3}$$

Par réc. : pour tout k , $q_k = a_n x^k + \dots + a_{n-k}$

En particulier, $q_n = P(x)$

Au final, cet algorithme est basé sur l'écriture suivante :

$$P(x) = (((\dots((a_n \times x + a_{n-1}) \times x + a_{n-2}) \dots) \times x + a_1) \times x + a_0$$

Nombre d'opérations : $2n$

Question 3 : Calculer à la main, par cette méthode, $P(4)$ avec $P = 2X^3 - 3X^2 - 2X + 5$.

	0	1	2	3
a_{n-k}	2	-3	-2	5
q_k	2	5	18	77
$q_k x$	8	20	72	

Question 4 : Compléter la fonction en Python qui programme cette méthode.

def horner(coeff, x):

```

q = coeff[-1]
for a in coeff[-2::-1]:
    q *= x
    q += a
return q

```

def horner(coeff, x):

```

q = coeff[-1]
n = len(coeff) - 1
for k in range(1, n+1):
    q = q * x + coeff[n-k]
return q

```